Hadoop java code cheatsheet

🧠 1. CORE INTUITION (MOST IMPORTANT)

Think of MapReduce like this:

🧩 Mapper = “Break + Tag”

- Reads input line
- Converts it into `(key, value)`

🔁 Shuffle = “Group by key”

- Hadoop automatically groups same keys

🔻 Reducer = “Process each group”

- Gets `(key, list of values)`
- Applies logic

🔥 Example (Word Count Idea)

```
Input:
A 10
A 20
B 30
```

Mapper:

```
(A,10), (A,20), (B,30)
```

Shuffle:

```
A → [10,20]
B → [30]
```

Reducer:

```
A → 30
B → 30
```

📄 2. UNIVERSAL TEMPLATE

✅ Mapper

```
public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

    String[] parts = value.toString().split(" ");
    String name = parts[0];
    int number = Integer.parseInt(parts[1]);

    // Apply logic
}
```

✅ Reducer

```
public void reduce(Text key, Iterable<IntWritable> values, Context
context)
        throws IOException, InterruptedException {

    for (IntWritable val : values) {
        int v = val.get();
    }
}
```

🔥 3. ALL PATTERNS (WITH INTUITION + CODE)

🟢 PATTERN 1: SUM

🧠 Intuition:

Add all values

Mapper

```
context.write(new Text("sum"), new IntWritable(number));
```

Reducer

```
int sum = 0;
for (IntWritable val : values) {
    sum += val.get();
}
context.write(key, new IntWritable(sum));
```

🔵 PATTERN 2: COUNT

🧠 Intuition:

Count number of records

Mapper

```
context.write(new Text("count"), new IntWritable(1));
```

Reducer

```
int count = 0;
for (IntWritable val : values) {
    count += val.get();
}
context.write(key, new IntWritable(count));
```

🔴 PATTERN 3: MAX

🧠 Intuition:

Track biggest value

Mapper

```
context.write(new Text("max"), new IntWritable(number));
```

Reducer

```
int max = Integer.MIN_VALUE;

for (IntWritable val : values) {
    if (val.get() > max)
        max = val.get();
}

context.write(key, new IntWritable(max));
```

🟣 PATTERN 4: MIN

🧠 Intuition:

Track smallest value

```
int min = Integer.MAX_VALUE;

for (IntWritable val : values) {
    if (val.get() < min)
```

```
            min = val.get();
    }
```

🟡 PATTERN 5: AVERAGE

🧠 Intuition:

Average = sum / count

```
int sum = 0, count = 0;

for (IntWritable val : values) {
    sum += val.get();
    count++;
}

context.write(key, new IntWritable(sum / count));
```

🟠 PATTERN 6: FILTER

🧠 Intuition:

Only pass valid records

```
if (number > 25000) {
    context.write(new Text(name), new IntWritable(number));
}
```

👉 Reducer not required

⚫ PATTERN 7: EVEN / ODD

```
if (number % 2 == 0) {
    context.write(new Text("even"), new IntWritable(1)); // count
}
```

🟤 PATTERN 8: SECOND MAX

🧠 Intuition:

Store → Sort → Pick second

```
```

```java
ArrayList<Integer> list = new ArrayList<>();

for (IntWritable val : values) {
    list.add(val.get());
}

Collections.sort(list, Collections.reverseOrder());

context.write(new Text("Second"), new IntWritable(list.get(1)));
```

🔮 PATTERN 9: MAX + MIN + DIFFERENCE

```java
int max = Integer.MIN_VALUE;
int min = Integer.MAX_VALUE;

for (IntWritable val : values) {
    int v = val.get();
    if (v > max) max = v;
    if (v < min) min = v;
}

context.write(new Text("Diff"), new IntWritable(max - min));
```

👩‍💻 4. JAVA BASICS (FOR NON-JAVA STUDENTS)

🔹 Important Classes

Type	Meaning
`Text`	String
`IntWritable`	Integer
`LongWritable`	Long

🔹 Convert Types

```java
value.toString()
Integer.parseInt(string)
val.get()
```

🔹 Loop

```java
for (IntWritable val : values)
```

👉 Means: loop through list

🔹 Array Split

```java
String[] parts = line.split(" ");
```

📦 5. JAVA COLLECTIONS YOU NEED

🔹 ArrayList

```java
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.get(0);
```

🔹 Sort

```java
Collections.sort(list);
Collections.sort(list, Collections.reverseOrder());
```

🔹 HashMap (Rare in exam)

```java
HashMap<String, Integer> map = new HashMap<>();
map.put("A", 10);
map.get("A");
```

🚀 6. DRIVER CODE (DON'T CHANGE MUCH)

```java
job.setMapperClass(MyMapper.class);
job.setReducerClass(MyReducer.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
```

⚠️ 7. COMMON MISTAKES

❌ Forgetting `.get()`
❌ Wrong data type
❌ Not using constant key for global operations
❌ Division by zero (avg)
❌ Index error in second max

🎯 8. EXAM STRATEGY

Step 1: Identify Pattern

Question says	Pattern
highest	MAX
lowest	MIN
total	SUM
count	COUNT
average	AVG
greater than	FILTER

Step 2: Decide KEY

Type	Key
Global	"sum"/"max"
Per user	name

Step 3: Write Loop

Every reducer = loop:

```
for (IntWritable val : values)
```

🧠 9. GOLDEN RULE

👉 If confused:

> “Put everything under one key and process in reducer”

⚡ 10. QUICK MEMORY FORMULA

```
Mapper → emit
Reducer → loop → compute → output
```

📄 🟢 PROGRAM 1: TOTAL SALARY (SUM PATTERN)

👉 Covers: Q3, Q12, Q17, Q10

```java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class TotalSalary {

    public static class MyMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

        public void map(LongWritable key, Text value, Context context)
                throws IOException, InterruptedException {

            String[] parts = value.toString().split(" ");
            int salary = Integer.parseInt(parts[1]);

            context.write(new Text("total"), new IntWritable(salary));
        }
    }

    public static class MyReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                throws IOException, InterruptedException {

            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }

            context.write(new Text("Total Salary"), new IntWritable(sum));
        }
    }

```

```
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Total Salary");

        job.setJarByClass(TotalSalary.class);
        job.setMapperClass(MyMapper.class);
        job.setReducerClass(MyReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

🔵 PROGRAM 2: COUNT RECORDS

👉 Covers: Q5, Q14, Q19

```
public static class MyMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

    public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

        context.write(new Text("count"), new IntWritable(1));
    }
}

public static class MyReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

        int count = 0;
        for (IntWritable val : values) {
            count += val.get();
        }

        context.write(new Text("Total Records"), new IntWritable(count));
    }
}
```

🔴 PROGRAM 3: MAX (HIGHEST SALARY)

👉 Covers: Q1, Q16, Q20

```
public static class MyMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

```

```
    public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

        String[] parts = value.toString().split(" ");
        int salary = Integer.parseInt(parts[1]);

        context.write(new Text("max"), new IntWritable(salary));
    }
}

public static class MyReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

        int max = Integer.MIN_VALUE;

        for (IntWritable val : values) {
            if (val.get() > max)
                max = val.get();
        }

        context.write(new Text("Max Salary"), new IntWritable(max));
    }
}
```

🟣 PROGRAM 4: MIN (LOWEST)

👉 Covers: Q2, Q11

```
public static class MyReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

        int min = Integer.MAX_VALUE;

        for (IntWritable val : values) {
            if (val.get() < min)
                min = val.get();
        }

        context.write(new Text("Min Salary"), new IntWritable(min));
    }
}
```

🟡 PROGRAM 5: AVERAGE

👉 Covers: Q4, Q8, Q13

```
public static class MyReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
```

```
2
3       public void reduce(Text key, Iterable<IntWritable> values, Context
    context)
4               throws IOException, InterruptedException {
5
6           int sum = 0, count = 0;
7
8           for (IntWritable val : values) {
9               sum += val.get();
10              count++;
11          }
12
13          context.write(key, new IntWritable(sum / count));
14      }
15  }
```

🟠 PROGRAM 6: FILTER (> 25000)

👉 Covers: Q6, Q15

```
1   public static class MyMapper extends Mapper<LongWritable, Text, Text,
    IntWritable> {
2
3       public void map(LongWritable key, Text value, Context context)
4               throws IOException, InterruptedException {
5
6           String[] parts = value.toString().split(" ");
7           String name = parts[0];
8           int salary = Integer.parseInt(parts[1]);
9
10          if (salary > 25000) {
11              context.write(new Text(name), new IntWritable(salary));
12          }
13      }
14  }
```

⚫ PROGRAM 7: EVEN COUNT

👉 Covers: Q9

```
1   public static class MyMapper extends Mapper<LongWritable, Text, Text,
    IntWritable> {
2
3       public void map(LongWritable key, Text value, Context context)
4               throws IOException, InterruptedException {
5
6           String[] parts = value.toString().split(" ");
7           int num = Integer.parseInt(parts[1]);
8
9           if (num % 2 == 0) {
10              context.write(new Text("even"), new IntWritable(1));
11          }
12      }
13  }
```

🟤 PROGRAM 8: SECOND HIGHEST

👉 Covers: Q7 (IMPORTANT)

```
import java.util.*;

public static class MyReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

        ArrayList<Integer> list = new ArrayList<>();

        for (IntWritable val : values) {
            list.add(val.get());
        }

        Collections.sort(list, Collections.reverseOrder());

        context.write(new Text("Second Highest"),
                      new IntWritable(list.get(1)));
    }
}
```

🧠 FINAL EXAM STRATEGY (SERIOUSLY IMPORTANT)

⚡ If stuck, do THIS:

1. Use **constant key**

```
context.write(new Text("key"), value);
```

2. Do everything in reducer:

```
for (IntWritable val : values)
```

3. Apply logic:

- sum
- max
- min
- count

🏁 LAST MINUTE REVISION

👉 You only need to remember:

- 1 Mapper pattern
- 1 Reducer loop
- 6 operations
